CanItShip: Closing the Feedback Loop on Agent Tooling via Trajectory Analysis and Scripted DevX Metrics

Evgenii Kniazev
Databricks
evgenii.kniazev@databricks.com

Arseny Kravchenko
Databricks
arseny.kravchenko@databricks.com

Fabian Jakobs
Databricks
fabian.jakobs@databricks.com

Igor Rekun
Databricks
igor.rekun@databricks.com

Abstract

When AI agents fail to build production applications, should we improve the agent or its environment? Recent work suggests that environment quality (templates, tools, guidance) matters more than model selection [11]. We present a feedback loop for iterative tooling improvement: (1) an *installable domain knowledge* architecture packages expertise as agent-consumable artifacts; (2) agents use this package to generate applications, producing execution trajectories; (3) an *agentic trajectory analyzer* processes these trajectories to identify friction and recommend fixes to the package; (4) *scripted DevX metrics* serve as the final quality gate, measuring whether generated applications can be run and deployed. We validate on Databricks data applications across two agent backends (Claude Code and Opencode+Sonnet). On 50 data-focused prompts requiring SQL, schema access, and platform integration, we achieve 100% build and type safety, 98% runtime success, and 96% local runability at $1.61/app—compared to 73.3% viability on easier consumer prompts in prior work. Extended checks on Opencode apps reveal that 77% pass runability and 54% pass deployment validation—apps passing core checks still fail on entry-point discovery and deployment configuration. The trajectory analyzer closes the loop: it processes failure traces, recommends fixes applied via Claude Code, and runability improves from 77% to 94% on the expanded set. The architecture targets artifacts amenable to automatic optimization [1, 9], potentially closing the loop without human intervention.

CCS Concepts

• **Software and its engineering → Software testing and debugging**; *Software development process management*.

Keywords

CanItShip, agent-agnostic tooling, agentic code generation, trajectory analysis, developer experience metrics, feedback loop

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
EASE '26, Glasgow, United Kingdom
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/06
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Evgenii Kniazev, Arseny Kravchenko, Fabian Jakobs, and Igor Rekun. 2026. CanItShip: Closing the Feedback Loop on Agent Tooling via Trajectory Analysis and Scripted DevX Metrics. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE '26), June 9–12, 2026, Glasgow, United Kingdom.* ACM, New York, NY, USA, 5 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

AI agents can generate functional software, but they lack domain-specific knowledge required for production applications. The conventional response is building specialized agents. We take a different approach: improve what agents have access to.

This approach is motivated by recent findings [11]: when comparing agent performance across environments while holding the model constant, environment quality (templates, tools, guidance) had larger effects than model upgrades. An agent with excellent tooling outperforms a better model with poor tooling.

Accepting that tooling matters raises a question: how do we improve it systematically? Manual inspection doesn't scale. End-state metrics (build pass/fail) don't reveal causes. We need a feedback loop that: (1) captures how agents actually use tooling; (2) identifies friction points and root causes; (3) produces actionable recommendations; (4) verifies improvements against agent-centric criteria.

We instantiate each component:

(1) **Installable Domain Knowledge** (Section 2). An architecture for packaging domain expertise as agent-consumable artifacts, exposing tools via CLI commands compatible with emerging skill-based standards.
(2) **Agentic Trajectory Analyzer** (Section 3). A two-phase system: parallel friction extraction with a fast model followed by agentic synthesis with a reasoning model that has source code access.
(3) **CanItShip: DevX Metrics** (Section 4). The final quality gate: eight shell-scripted metrics (**Ls1–Ls8**) covering build correctness, runtime health, framework-adaptive runability, and deployment validation.

Validation. We validate across Claude Code (primary, n=47 of 50 prompts) and Opencode+Sonnet (n=35 of 50) on Databricks data applications; gaps are due to generation failures and non-standard project structures (Table 2). On 47 Claude Code applications: 100% build and type safety, 98% runtime success, and 96% local runability

at $1.61/app average. The trajectory analyzer identified improvements that we implemented and verified through multiple iterations.

2 Installable Domain Knowledge

Developers already use capable agents (Claude Code, Opencode [15]). Rather than asking them to adopt yet another tool, we bring domain knowledge to the agents they already use. A user installs our package into their existing environment; the agent gains domain expertise without workflow changes.

The package has three components: context layers that inject domain knowledge progressively, tools exposed via CLI, and a state machine that enforces validation before deployment.

Context layers. Agents have limited context windows. Dumping all domain knowledge upfront wastes tokens and can confuse the model. Instead, we inject context progressively: tool definitions are always available at the protocol level, workflow patterns activate on first discovery call, target-specific constraints load when the application type is detected, and SDK-specific templates inject after scaffolding.

CLI tools and MCP parity. We expose domain functionality through CLI commands, a pattern that Cloudflare [6] and Anthropic [3] found effective. Each CLI command maps 1:1 to a Model Context Protocol (MCP) tool definition, ensuring the same domain knowledge is accessible to any MCP-compliant agent. Lifecycle commands (`scaffold`, `validate`, `deploy`) and data exploration commands (schema discovery, batch queries) are available as both CLI tools and MCP skills, providing the domain-specific capabilities on top of standard workspace tools.

State machine. Applications cannot deploy unless they pass validation after their most recent modification: `Scaffolded` → `Modified` → `Validated` → `Deployed`. Any change after validation requires re-validation, preventing untested code deployment, a common failure mode when agents skip validation.

Agent compatibility. To validate agent-agnosticism, we tested the package across Claude Code (automated, primary) and Opencode with Sonnet. The key requirement is function calling capability; any agent that can invoke CLI tools works with our package.

3 Agentic Trajectory Analyzer

The analyzer consumes execution trajectories and recommends package improvements. This closes the error analysis loop [4]: agents struggle → we see it in trajectories → we fix the tooling → agents struggle less.

End-state metrics (build success, test pass) don't reveal whether failures stem from model limitations, tool problems, missing domain knowledge, or underspecified prompts. Trajectories (the sequence of reasoning, tool calls, and results) show where things went wrong. An agent retrying the same malformed SQL five times reveals a missing example. An agent calling N tools for N tables reveals a missing batch operation.

Two-phase architecture. We employ a map-reduce approach optimized for cost and quality. In the *map phase*, each trajectory is processed independently by a fast model (Claude Haiku, ~ $0.001 per trajectory), extracting errors, retries, confusion patterns, and inefficient tool usage. This runs in parallel. In the *agentic synthesis phase*, aggregated patterns go to a reasoning model (Claude Opus) with read-only access to template source code, tool definitions, and prior analysis history, generating prioritized recommendations.

Cost model. For N trajectories: map costs $N \times$ ~$0.001 (fast model), synthesis costs ~$0.5–3 (reasoning model, bounded at 50 turns). For 50 apps, total analysis cost was under $15.

Human judgment on small N. On small datasets, we found that the analyzer can overreact, encountering an issue once or twice and recommending building a helper utility without assessing maintenance cost. We recommend human review of recommendations when $N < 50$, treating the analyzer output as a prioritized error analysis rather than an implementation directive.

4 DevX Metrics

DevX evaluation is the final quality gate. After the package is improved and agents generate applications, we need to verify: can these applications be run and deployed?

The CanItShip framework. All eight metrics (Table 1) produce binary pass/fail verdicts from deterministic commands, spanning three tiers: Core Functionality (Ls1–Ls4), Platform Integration (Ls5), and DevX (Ls6–Ls8). Ls1–Ls6 target the primary agent stack (Node.js/TypeScript for Claude Code). Ls7–Ls8 extend coverage to Python/Streamlit (Opencode): Ls7 detects the framework and entry point before running a 3-trial install-start-healthcheck protocol, while Ls8 runs `databricks bundle validate`. Non-deterministic metrics (Ls6, Ls7) pass if at least two of three trials succeed. All metrics are shell-scripted, ensuring reproducibility (see Discussion for our experience with agentic alternatives).

Table 1: CanItShip metric definitions. All metrics are shell-scripted (Ls).

ID	Metric	Check
Ls1	Build	Build scripts / import sanity
Ls2	Runtime	Start app, healthcheck on port
Ls3	Types	Static analysis (pyright, tsc)
Ls4	Tests	Run test suite; FAIL if none
Ls5	UI	Render page, check for errors
Ls6	Runability	Install, start, healthcheck
Ls7	Runability	Framework-adaptive install+start
Ls8	Deployability	`databricks bundle validate`

Generation–evaluation isolation. Agents that can read evaluation scripts produce code that passes checks rather than solving the task—a form of reward hacking [2]. We enforce workspace-level isolation, excluding evaluation scripts from the generation environment. Each generation starts from a clean workspace, mirroring ML train/test splits.

5 Results

We organize results around three claims: (1) automated scripted evaluation on a new data-focused benchmark; (2) extended checks revealing failures invisible to core metrics; (3) a trajectory-driven closed loop that analyzes failures, applies fixes, and measures improvement.

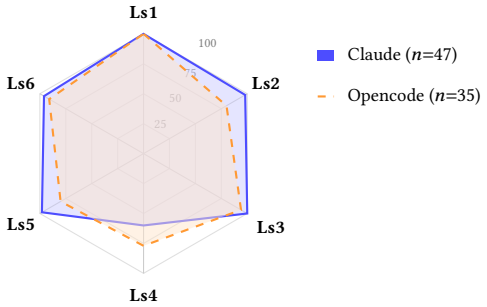

Figure 1: Shell-scripted metric profiles (Ls1–Ls6). Both agents achieve near-perfect build and type-check scores; the main gaps are Ls2 (runtime), Ls4 (tests), and Ls5 (UI). Extended metrics (Ls7–Ls8) evaluated on Opencode Set 1 are in Table 5.

Table 2: Dataset coverage. Set 1: 20 data analytics prompts. Set 2: 30 prompts with additional platform integration complexity (deployment configs, multi-service architectures). Generation failures produced only metadata with no source code; Ls-evaluated counts further exclude apps whose non-standard project structures prevented the harness from discovering entry points.

	Gen.	Gen. fail	Ls1–6	Ls7–8
Claude (Set 1+2)	49/50	1	47	—
Opencode (Set 2)	28/30	2	22	—
Opencode (Set 1)	18/20	2	13	13→18

Claim 1: Automated scripted evaluation on new data-focused benchmark. Where app.build [11] achieved 73.3% viability on 30 generic consumer prompts with human evaluation, we create 50 new data-focused prompts requiring SQL, schema access, and platform integration, and automate evaluation as reproducible shell scripts. Two disjoint prompt sets were given to both agents under identical conditions (Table 2). Shell-scripted checks (Ls1–Ls6) ran on both agents to measure how the harness interacts with each backend. Extended checks (Ls7–Ls8) ran on Opencode Set 1 apps only; 5 were initially excluded because their non-standard project structures prevented the harness from discovering entry points.

Shell-scripted metrics (Ls1–Ls6) were evaluated using deterministic build, runtime, type-check, test, UI, and HTTP probe commands with a 3-trial majority protocol for Ls6.

The 100% build and type-safety rates across all 47 apps confirm that Claude Code with installable domain knowledge produces structurally sound applications. Ls2 (runtime) and Ls5 (UI) each achieve 98%, with a single app failing to respond on the healthcheck endpoint. Ls6 (runability) reaches 96% under the 3-trial majority protocol. Ls4 (tests) shows 60%, as 19 app test suites contain integration tests that require a live database connection unavailable in local evaluation.

Multi-agent comparison. To validate agent-agnosticism, we ran the same shell-scripted harness on 35 Opencode+Sonnet applications (50 prompts; 4 generation failures, 11 non-standard structures) across both prompt sets (Table 4). Both agents achieve 100% build (Ls1); divergence appears on runtime, UI, and DevX metrics. The 98% runtime for Claude Code versus 80% for Opencode reflects two factors: architectural differences (Node.js apps expose consistent healthcheck endpoints, while Python/Streamlit apps require framework-specific entry-point detection) and maturity of domain knowledge integration (the Claude Code package has undergone more iteration cycles, as the trajectory analyzer was primarily run on Claude Code trajectories first).

Table 3: Shell-scripted evaluation (Ls metrics, n=47 Claude Code apps out of 50 prompts; 1 generation failure, 2 excluded due to non-standard project structure).

Tier	Metric	Result
Core	Ls1: Build Success	47/47 (100%)
	Ls2: Runtime Success	46/47 (98%)
	Ls3: Type Safety	47/47 (100%)
	Ls4: Tests Pass	28/47 (60%)
Platform	Ls5: UI Renders	46/47 (98%)
DevX	Ls6: Runability	45/47 (96%)
Efficiency	Cost per App	$1.61
	Turns per App	46 avg

Extended metrics (Ls7–Ls8) in Table 5.

Table 4: Multi-agent shell-scripted (Ls) comparison: Claude (n=47) vs Opencode (n=35).

Metric	Claude	Opencode	Δ
Ls1: Build	100%	100%	0
Ls2: Runtime	98%	80%	+18
Ls3: Types	100%	94%	+6
Ls4: Tests	60%	77%	−17
Ls5: UI	98%	80%	+18
Ls6: Runability	96%	91%	+5

Claim 2: Extended checks—can these apps be run and deployed? Ls7–Ls8 test whether the harness generalizes beyond the Claude Code Node.js stack to Opencode Python apps (n=18 evaluable, 2 excluded as empty generation failures; Table 5).

Table 5: Extended evaluation (Ls7–Ls8): Opencode+Sonnet Set 1. Before: n=13 discoverable apps. After: fixes expand to n=18.

Metric	Before	After	Δ	Failure
Ls7	10/13 (77%)	17/18 (94%)	+17pp	No entry point
Ls8	7/13 (54%)	10/18 (56%)	+2pp	No deploy config

Before fixes, Ls7 passed at 77%—three apps were generated as batch libraries without web entry points. Ls8 failures stemmed from apps lacking `databricks.yml` entirely or having invalid bundle configurations.

Ls6 runability variance. Even deterministic scripts exhibit nondeterminism from timing and port contention. Under the 3-trial

majority protocol, Claude Code yields 96% (38 apps 3/3, 7 apps 2/3, 2 apps 0/3) and Opencode yields 91% (24 apps 3/3, 8 apps 2/3, 3 apps 0/3). Multi-trial protocols are essential for metrics involving process startup and network probes.

Claim 3: Trajectory analysis closes the loop. The trajectory analyzer processes failure traces from Ls7–Ls8, recommends fixes, and we apply them via Claude Code and rerun. Table 6 shows manual loop iterations; these fixes—targeting tool descriptions, prompts, and examples—are amenable to automatic optimization [1, 9].

Table 6: Trajectory-identified improvements (manual loop iterations).

Pattern Found	Root Cause	Fix Applied
N calls for N tables	Missing batch op	Added batch schema discovery
Repeated SQL errors	Missing examples	Added QUALIFY, PIVOT to guidance
Confusing tool name	Name mismatch	Renamed tool for clarity
Retries on errors	Unclear messages	Added contextual error params
6% type safety	Missing null examples	Added null handling to templates

The before/after columns in Table 5 show the closed loop: the trajectory analyzer identified that three apps were batch libraries without web entry points and six lacked deployment configurations. After applying fixes (Streamlit wrappers for batch apps, root-level dependency files for five previously unevaluable apps), runability improved from 77% to 94% on the expanded set.

6 Related Work

Environment over model. Recent work [11] found that environment quality had larger effects than model upgrades when comparing agent performance. SWE-agent [18] similarly demonstrated that agent-computer interfaces significantly affect performance. This motivates our focus on improving tooling rather than agents.

Evaluation gap. Existing benchmarks evaluate code correctness (HumanEval [5], SWE-bench [8]), task completion (WebArena [20], GAIA [14]), or SQL quality (BIRD [12], Spider [19]). None ask whether generated code can be operated by other agents, a critical question for compound AI systems.

Trajectory analysis. Analyzing agent execution traces for debugging is established practice (SWE-agent [18], OpenHands [17]). Our contribution is making the synthesis phase itself agentic: an agent that explores source code to generate recommendations, rather than static aggregation.

Agent evaluation harnesses. Inspect AI [16] and DeepEval [7] provide evaluation infrastructure but focus on correctness rather than deployment readiness. AgentBench [13] spans multiple environments but does not evaluate autonomous deployability.

7 Discussion

Agent-agnostic vs agent-specific. Our approach invests in tooling rather than agents: agents improve rapidly (model upgrades are free to adopters), while domain knowledge is slow to encode and hard to transfer. The tradeoff is control; agent-specific approaches can fine-tune behavior precisely, while we rely on the ability of agents to follow tool descriptions. In practice, this suffices for structured tasks but limits open-ended architectural reasoning.

Scripted vs. agentic evaluation. We initially prototyped an agentic evaluator (Claude Haiku autonomously running and deploying each app). Agentic evaluation introduced measurement noise—the LLM chose different commands across runs, wrapped status markers in markdown that broke parsers, and produced false positives—conflating evaluator defects with genuine app defects. It was also slow (sequential due to port conflicts) and expensive (token cost per check). However, the prototype was invaluable for *discovering* what to script: it surfaced framework-detection needs and deployment-validation patterns that we encoded as Ls7–Ls8. Prior work [10] found that simple, reliable baseline metrics serve as firm ground for reproducible evaluation across large code datasets. Inspired by this, we shifted to a limited set of metrics directly and reproducibly derived from the code—deterministic, parallelizable, and free of LLM cost—reserving agentic evaluation for exploratory settings where evaluation criteria are still being defined.

Toward automatic optimization. Our loop is currently semi-automatic: the analyzer outputs recommendations, a human reviews them. Since recommendations target tool descriptions and prompts, techniques like GEPA [1] and DSPy [9] could potentially close the loop without human intervention.

Limitations. *External validity:* We validate on Databricks; the harness is platform-independent but domain packages require adaptation. *Internal validity:* Shell metrics cover n=47 Claude and n=35 Opencode apps; extended metrics (Ls7–Ls8) cover n=18 Opencode Set 1. Scaling to 100+ is planned. *Construct validity:* Even deterministic scripts exhibit timing variance; multi-trial protocols are essential. The feedback loop is demonstrated via case studies (Table 6); longitudinal quantification is planned.

8 Conclusion

We presented a feedback loop for iterative improvement of agent tooling: installable domain knowledge, an agentic trajectory analyzer, and CanItShip metrics. Using shell-scripted evaluation (Ls1–Ls8) across 82 Databricks applications and two agent backends (Claude Code n=47 of 50 prompts, Opencode+Sonnet n=35 of 50; remainder excluded due to generation failures and non-standard structures), we achieve 100% build and type safety, 98% runtime success, and 96% local runability at $1.61/app. Extended checks on Opencode apps (n=18) show 77% runability and 54% deployment validation before fixes, with failures traceable to missing entry points and deployment manifests. We demonstrate the closed loop: the trajectory analyzer identifies failure patterns, fixes are applied via Claude Code, and runability improves from 77% to 94%. When agents fail, improve their environment rather than the agents themselves. The evaluation harness, trajectory analyzer, 50-prompt benchmark, and anonymized artifacts are available as open source.[1]

Acknowledgments

We thank the Databricks Apps team for infrastructure support.

[1] https://github.com/neondatabase/appdotbuild-agent

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *arXiv preprint arXiv:2507.19457* (2025).

[2] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565* (2016).

[3] Anthropic. 2024. Tool Use with Claude. https://docs.anthropic.com/en/docs/build-with-claude/tool-use.

[4] Valerii Babushkin and Arseny Kravchenko. 2025. *Machine Learning System Design: With End-to-End Examples.* Manning Publications, Chapter 9.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[6] Cloudflare. 2025. Code Mode: Give Agents an Entire API in 1,000 Tokens. https://blog.cloudflare.com/code-mode-mcp/.

[7] Confident AI. 2024. DeepEval: The LLM Evaluation Framework. https://github.com/confident-ai/deepeval.

[8] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *arXiv preprint arXiv:2310.06770* (2024).

[9] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv preprint arXiv:2310.03714* (2023).

[10] Evgenii Kniazev. 2026. Automated Classification of Source Code Changes Based on Metrics Clustering in the Software Development Process. *arXiv preprint arXiv:2602.14591* (2026).

[11] Evgenii Kniazev, Arseny Kravchenko, Igor Rekun, James Broadhead, Nikita Shamgunov, Pranav Sah, Pratik Nichite, and Ivan Yamshchikov. 2025. app.build: A Production Framework for Scaling Agentic Prompt-to-App Generation with Environment Scaffolding. arXiv:2509.03310 [cs.AI]

[12] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQL. *Advances in Neural Information Processing Systems* 36 (2023).

[13] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. AgentBench: Evaluating LLMs as Agents. *arXiv preprint arXiv:2308.03688* (2023).

[14] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: A Benchmark for General AI Assistants. *arXiv preprint arXiv:2311.12983* (2023).

[15] Opencode. 2025. Opencode: The Open Source Coding Agent. https://opencode.ai/.

[16] UK AI Safety Institute. 2024. Inspect: A Framework for Large Language Model Evaluations. https://inspect.aisi.org.uk/.

[17] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741* (2024).

[18] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793* (2024).

[19] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *arXiv preprint arXiv:1809.08887* (2018).

[20] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv preprint arXiv:2307.13854* (2024).